

Mini Assessment

Name: Mangali Bhaskar

Reg No: 99230040657

Dept: CSE AIML

1. If an API returns 429, what should your program do?

Answer: A 429 response means the server is temporarily refusing requests because the allowed request rate has been exceeded. The application should pause before trying again rather than repeatedly calling the API.

- Read the Retry-After header when the server provides one.
- Retry after increasing delays, such as 2, 4, 8 and 16 seconds.
- Set a maximum number of retries to avoid an endless loop.
- Record the failure and show a suitable message if the request still cannot be completed.

Example: A news application making many requests may receive 429. It can delay the next request and continue after the rate limit period.

2. If response.json() fails, what should your program do?

Answer: The program should treat the response as possibly non-JSON and handle the parsing failure with exception handling. It should first consider the HTTP status and the actual response content before trying to use JSON data.

- Catch the JSON decoding exception with try/except.
- Check whether the response contains the expected content type.
- Record enough information to diagnose the problem.
- Provide a safe fallback instead of allowing the application to terminate.

Example: A server might return a plain-text or HTML error page instead of JSON. Calling response.json() on that content can fail, so the error should be handled safely.

3. If a JSON key is missing, why might .get() be safer than []?

Answer: Using data['key'] expects the key to exist and raises an error when it does not. The .get() method is safer when a field is optional because it can return a default value instead.

- Avoids a KeyError for an absent field.
- Allows the program to specify a meaningful fallback value.
- Works well when API responses may contain optional properties.
- Makes the code more defensive when processing external data.

Example: data.get('email', 'No email available') can continue even when the email field is absent.

Conclusion

Good API programs should handle server limits, unexpected response formats, and incomplete JSON data gracefully. Using controlled retries, exception handling, and safe access to optional fields helps keep applications dependable.